

Операторы в SCSS

Что такое операторы в SCSS?

Операторы в SCSS позволяют выполнять математические и логические операции непосредственно в таблицах стилей. Это делает SCSS более мощным и гибким, позволяя создавать динамические значения и условную логику.

Математические операторы

SCSS поддерживает стандартные математические операции:

```
// Сложение
.element {
  width: 100px + 50px; // 150px
}

// Вычитание
.element {
  margin: 20px - 5px; // 15px
}

// Умножение
.element {
  width: 5 * 10px; // 50px
}

// Деление
.element {
  width: (100px / 2); // 50px
}

// Остаток от деления (модуль)
.element {
  z-index: 11 % 3; // 2
}
```

Особенности математических операций

Единицы измерения

При выполнении математических операций SCSS учитывает единицы измерения:

```
// Совместимые единицы
.element {
  font-size: 16px + 4px; // 20px
}
```

```

margin: 5em - 1em; // 4em
padding: 5rem * 2; // 10rem
width: 100px / 2; // 50px
}

// Несовместимые единицы вызовут ошибку
.element {
  // Ошибка: несовместимые единицы px и em
  // width: 100px + 1em;
}

```

Умножение и деление с единицами измерения

```

.element {
  // При умножении единицы перемножаются
  // 5px * 5px = 25px*px (ошибка)

  // При делении единицы сокращаются
  // 100px / 5px = 20 (без единиц)
  font-size: (100px / 5); // 20px
}

```

Скобки для приоритета операций

```

.element {
  // Без скобок: 100px / 2 + 10px = 100px / (2 + 10px) = 100px / 12px = 8.33
  // Со скобками: (100px / 2) + 10px = 50px + 10px = 60px
  width: (100px / 2) + 10px; // 60px
}

```

Строковые операторы

Конкатенация строк

Для объединения строк в SCSS используется оператор `+`:

```

.element {
  // Конкатенация строк без пробелов
  content: "Hello" + "World"; // "HelloWorld"

  // Конкатенация строк с пробелами
  font-family: "Helvetica" + " " + "Neue"; // "Helvetica Neue"

  // Конкатенация строк с кавычками и без
}

```

```
content: "Quote: " + Hello; // "Quote: Hello"
}
```

Операторы сравнения

SCSS поддерживает следующие операторы сравнения:

```
$a: 10;
$b: 5;

// Равно
$result1: $a == $b; // false

// Не равно
$result2: $a != $b; // true

// Больше
$result3: $a > $b; // true

// Меньше
$result4: $a < $b; // false

// Больше или равно
$result5: $a >= $b; // true

// Меньше или равно
$result6: $a <= $b; // false
```

Логические операторы

SCSS поддерживает логические операторы для комбинирования условий:

```
$a: true;
$b: false;

// Логическое И (and)
$result1: $a and $b; // false

// Логическое ИЛИ (or)
$result2: $a or $b; // true

// Логическое НЕ (not)
$result3: not $a; // false
$result4: not $b; // true

// Комбинирование операторов
$result5: $a and not $b; // true
$result6: not ($a or $b); // false
```

Использование операторов в условных выражениях

```
$theme: 'light';
$primary-color: #3498db;
$text-color: #333;

.button {
  @if $theme == 'dark' {
    background-color: lighten($primary-color, 10%);
    color: white;
  } @else {
    background-color: $primary-color;
    color: $text-color;
  }
}

// Использование логических операторов
@if $theme == 'light' and $primary-color == #3498db {
  border: 1px solid darken($primary-color, 10%);
}

// Использование математических операторов в условиях
$width: 100px;
@if $width > 50px and $width <= 150px {
  padding: $width / 10;
}
}
```

Интерполяция и операторы

Операторы можно использовать внутри интерполяции `#{}:`

```
$base: 'hello';
$suffix: 'world';
$num: 5;

.#{ $base }-#{ $suffix } {
  content: "Value: #{ $num * 2 }";
  width: #{ $num * 10 }px;
}

// Компилируется в:
// .hello-world {
//   content: "Value: 10";
//   width: 50px;
// }
```

Операторы в функциях и миксинах

```
// Функция с использованием операторов
@function calculate-width($base-width, $columns, $gutter) {
  @return $base-width * $columns + $gutter * ($columns - 1);
}

// Миксин с использованием операторов
@mixin grid-column($columns, $total-columns: 12) {
  width: percentage($columns / $total-columns);
  margin-right: if($columns == $total-columns, 0, 2%);
}

.container {
  max-width: calculate-width(100px, 3, 20px); // 340px
}

.sidebar {
  @include grid-column(4); // width: 33.33333%; margin-right: 2%;
}

.content {
  @include grid-column(8); // width: 66.66667%; margin-right: 2%;
}

.full-width {
  @include grid-column(12); // width: 100%; margin-right: 0;
}
```

Практические примеры использования операторов

Создание системы сетки

```
$columns: 12;
$gutter: 20px;
$container-width: 1200px;

@function column-width($cols) {
  $total-gutter-width: $gutter * ($columns - 1);
  $column-width: ($container-width - $total-gutter-width) / $columns;

  @if $cols == $columns {
    @return $cols * $column-width + $gutter * ($cols - 1);
  } @else {
    @return $cols * $column-width + $gutter * ($cols - 1);
  }
}

@for $i from 1 through $columns {
```

```

.col-#{ $i } {
  width: column-width( $i );

  @if $i < $columns {
    margin-right: $gutter;
  }
}
}

```

Создание цветовой палитры

```

$base-color: #3498db;
$steps: 5;
$step-percentage: 10%;

@for $i from 1 through $steps {
  .color-lighten-#{ $i } {
    background-color: lighten( $base-color, $i * $step-percentage );
  }

  .color-darken-#{ $i } {
    background-color: darken( $base-color, $i * $step-percentage );
  }
}

```

Создание типографической шкалы

```

$base-font-size: 16px;
$ratio: 1.25;

@for $i from -2 through 6 {
  .text-#{ $i + 2 } {
    font-size: $base-font-size * pow( $ratio, $i );
  }
}

// Вспомогательная функция для возведения в степень
@function pow( $base, $exponent ) {
  $result: 1;

  @if $exponent > 0 {
    @for $i from 1 through $exponent {
      $result: $result * $base;
    }
  } @else if $exponent < 0 {
    @for $i from 1 through -$exponent {
      $result: $result / $base;
    }
  }
}

```

```
}  
  
@return $result;  
}
```

Лучшие практики использования операторов

1. Используйте скобки для ясности

- Даже если они не всегда необходимы, скобки делают код более читаемым
- `width: (100% / 3);` понятнее, чем `width: 100% / 3;`

2. Будьте осторожны с единицами измерения

- Помните о совместимости единиц при выполнении операций
- Используйте функции для преобразования единиц при необходимости

3. Используйте переменные для сложных вычислений

- Сохраняйте промежуточные результаты в переменных для улучшения читаемости

4. Создавайте функции для повторяющихся вычислений

- Если вы часто выполняете одни и те же вычисления, создайте функцию

5. Избегайте слишком сложных выражений

- Разбивайте сложные выражения на более простые части
- Используйте комментарии для объяснения сложной логики

Заключение

Операторы в SCSS - это мощный инструмент, который позволяет создавать динамические, вычисляемые значения и условную логику в ваших стилях. Они особенно полезны для создания гибких систем дизайна, таких как сетки, цветовые палитры и типографические шкалы. Правильное использование операторов может значительно улучшить организацию вашего кода, уменьшить дублирование и сделать стили более гибкими и поддерживаемыми.